Firebase Forum & Support System Setup Guide

Firestore Collections Structure

1. forum_posts

javascript

```
{
  id: "auto-generated-id",
  question: "How do I upgrade to Premium?",
  author_id: "user-uid-here",
  created_at: Timestamp
}
```

2. forum_answers

javascript

```
{
  id: "auto-generated-id",
  post_id: "forum-post-id",
  text: "Click Upgrade in Settings",
  author_id: "user-uid-here",
  created_at: Timestamp
}
```

3. support_requests

javascript

```
{
  id: "auto-generated-id",
  user_id: "user-uid-here",
  message: "I need help with my account",
  created_at: Timestamp
}
```

4. users (for admin role)

javascript

```
{
  id: "user-uid",
  role: "admin" // or omit for regular users
}
```

Setup Instructions

Step 1: Add Security Rules to Firebase

1. Go to Firebase Console
2. Select your project: **good-energy-8b1b4**
3. Click **Firestore Database** in the left menu
4. Click the **Rules** tab at the top
5. **Copy the entire content** from the "Firebase Firestore Security Rules" artifact above
6. **Paste it** into the rules editor (replace everything)
7. Click **Publish**

Step 2: Create Firestore Indexes

Some queries require indexes. Firebase will tell you which ones when you run the app, or you can create them manually:

1. In Firestore Database → **Indexes** tab
2. Create these composite indexes:

Index 1: forum_answers

- Collection: `forum_answers`
- Fields: `post_id` (Ascending), `created_at` (Ascending)

Index 2: forum_posts

- Collection: `forum_posts`
- Fields: `created_at` (Descending)

Index 3: support_requests

- Collection: `support_requests`
- Fields: `created_at` (Descending)

Step 3: Make Your First Admin User

After you create your account in the app, you need to manually set yourself as admin:

1. Go to Firestore Database → **Data** tab
2. Click **Start collection**
3. Collection ID: `users`
4. Document ID: **your-user-uid** (get this from Authentication → Users)
5. Add field:
   - Field: `role`
   - Type: `string`
   - Value: `admin`
6. Click **Save**

Or use the Firebase Console in your browser's developer tools:

```javascript
import { doc, setDoc } from 'firebase/firestore';
await setDoc(doc(db, 'users', 'YOUR-UID-HERE'), { role: 'admin' });
```

Key Differences from Supabase

Feature	Supabase (SQL)	Firebase (NoSQL)
Data Structure	Relational tables with foreign keys	Collections of documents
Queries	SQL with JOINs	JavaScript queries, no joins
Security	RLS policies with SQL	Security rules with custom functions
Auto-incrementing IDs	BIGINT with sequences	Auto-generated strings
Timestamps	`timestamptz`	`serverTimestamp()`

Feature	Supabase (SQL)	Firebase (NoSQL)
Cascading Deletes	ON DELETE CASCADE	Manual or Cloud Functions

Important Notes:

1. **No Cascading Deletes**: In Firestore, when you delete a `forum_post`, the associated `forum_answers` don't automatically delete. You need to:
   - Delete them manually in your code
   - Or use Firebase Cloud Functions to handle it
2. **No JOINs**: To get a forum post with its answers, you need two queries:

```javascript
// Get post
const post = await getDoc(doc(db, 'forum_posts', postId));

// Get answers separately
const answers = await getForumAnswers(postId);
```

3. **Author Information**: To display author names, you'll need to:
   - Query the `profiles` collection separately
   - Or denormalize by storing author usernames in posts/answers

---

Usage Example in Your App

```javascript
```

```
import { createForumPost, getForumPosts, createForumAnswer, getForumAnswers } from './forumFunctions';

// In your component
const ForumComponent = () => {
  const [posts, setPosts] = useState([]);

  useEffect(() => {
    loadPosts();
  }, []);

  const loadPosts = async () => {
    const result = await getForumPosts();
    if (result.success) {
      setPosts(result.posts);
    }
  };

  const handleCreatePost = async (question) => {
    const result = await createForumPost(question);
    if (result.success) {
      loadPosts(); // Refresh posts
    }
  };

  return (
    <div>
      {posts.map(post => (
        <ForumPost key={post.id} post={post} />
      ))}
    </div>
  );
};
```

Testing Your Setup

1. **Test Forum Posts**: Create a post as a logged-in user
2. **Test Answers**: Reply to a forum post
3. **Test Permissions**: Try to edit someone else's post (should fail)
4. **Test Support**: Submit a support request

5. **Test Admin**: Make yourself admin and view support requests

---

Troubleshooting

Error: "Missing or insufficient permissions"

- Check that your security rules are published
- Verify the user is authenticated
- Check that `author_id` matches the current user's UID

Error: "The query requires an index"

- Click the link in the error message to auto-create the index
- Or manually create indexes as described in Step 2

Support requests not visible

- Make sure you set your user's role to "admin" in the `users` collection
- The security rules check for `role == 'admin'`